부록 · LangChain4j · Spring AI

Spring Boot Java + Kotlin 기술지표·Gemma3 실전 예제

Wilder RSI · MA · EMA · MACD · 볼린저밴드 · 거래량 → LangChain4j → Ollama Gemma3

```yaml
Ollama: http://localhost:11434
```

1. 권장 전체 구조

```text
업비트 KRW-XRP 60분봉 200개
        ↓
Spring Boot / IndicatorService
 ├─ MA5 / MA20
 ├─ EMA12 / EMA26
 ├─ Wilder RSI14
 ├─ MACD(12,26,9)
 ├─ Bollinger Band(20, 2σ)
 └─ Volume MA20
        ↓
IndicatorDto (계산된 숫자만 정리)
        ↓
LangChain4j
        ↓
Ollama → Gemma3
        ↓
상승 / 하락 / 횡보 + 지표별 해석
```

핵심 원칙: 200개 캔들을 LLM에 그대로 보내지 않고 Java/Kotlin이 수치 계산을 담당하며, Gemma3는 계산된 지표를 해석하는 역할만 담당합니다.

2. Java와 Kotlin 구성 비교

구성	Java	Kotlin
데이터 객체	record	data class
최근 N개	subList()	takeLast()
EMA 반복	for 루프	forEach 가능
MACD 결과	double[]	Triple
계산 담당	IndicatorService	IndicatorService
AI 해석	XrpAiService	XrpAiService

3. IndicatorDto

Java - IndicatorDto.java

```text
public record IndicatorDto(
    double price,
    double ma5,
    double ma20,
    double ema12,
    double ema26,
    double rsi14,
    double macd,
    double macdSignal,
    double macdHistogram,
    double bollingerUpper,
    double bollingerMiddle,
    double bollingerLower,
    double volume,
    double volumeMa20
) {}
```

Kotlin - IndicatorDto.kt

```kotlin
data class IndicatorDto(
    val price: Double,
    val ma5: Double,
    val ma20: Double,
    val ema12: Double,
    val ema26: Double,
    val rsi14: Double,
    val macd: Double,
    val macdSignal: Double,
    val macdHistogram: Double,
    val bollingerUpper: Double,
    val bollingerMiddle: Double,
    val bollingerLower: Double,
    val volume: Double,
    val volumeMa20: Double
)
```

4. 지표의 뜻

지표	기본 설정	쉬운 의미
MA	5, 20	최근 종가의 단순 평균
EMA	12, 26	최근 가격에 더 큰 가중치를 둔 평균
Wilder RSI	14	상승폭과 하락폭의 상대 강도
MACD	12,26,9	EMA12-EMA26 및 Signal과의 차이
볼린저밴드	20, 2σ	MA20 기준 변동성 범위
거래량 MA	20	현재 거래량과 최근 평균 거래량 비교

5. Java - IndicatorService 핵심 함수

```java
@Service
public class IndicatorService {

    public double ma(List<CandleDto> candles, int period) {
        return candles.subList(candles.size() - period, candles.size())
                .stream()
                .mapToDouble(CandleDto::tradePrice)
                .average()
                .orElse(0);
    }

    public double ema(List<CandleDto> candles, int period) {
        double k = 2.0 / (period + 1);
        double ema = candles.get(0).tradePrice();

        for (int i = 1; i < candles.size(); i++) {
            double price = candles.get(i).tradePrice();
            ema = price * k + ema * (1 - k);
        }
        return ema;
    }

    public double wilderRsi(List<CandleDto> candles, int period) {
        if (candles.size() <= period) return 0;

        double gain = 0, loss = 0;

        for (int i = 1; i <= period; i++) {
            double change = candles.get(i).tradePrice()
                    - candles.get(i - 1).tradePrice();
            if (change > 0) gain += change;
            else loss += Math.abs(change);
        }

        double avgGain = gain / period;
        double avgLoss = loss / period;

        for (int i = period + 1; i < candles.size(); i++) {
            double change = candles.get(i).tradePrice()
                    - candles.get(i - 1).tradePrice();

            double currentGain = Math.max(change, 0);
            double currentLoss = Math.max(-change, 0);

            avgGain = (avgGain * (period - 1) + currentGain) / period;
            avgLoss = (avgLoss * (period - 1) + currentLoss) / period;
        }

        if (avgLoss == 0) return 100;
        double rs = avgGain / avgLoss;
        return 100 - (100 / (1 + rs));
    }

    public List<Double> emaSeries(List<Double> values, int period) {
        List<Double> result = new ArrayList<>();
```

```
        double k = 2.0 / (period + 1);
        double ema = values.get(0);
        result.add(ema);

        for (int i = 1; i < values.size(); i++) {
            ema = values.get(i) * k + ema * (1 - k);
            result.add(ema);
        }
        return result;
    }

    public double[] macd(List<CandleDto> candles) {
        List<Double> prices = candles.stream()
                .map(CandleDto::tradePrice).toList();

        List<Double> ema12 = emaSeries(prices, 12);
        List<Double> ema26 = emaSeries(prices, 26);
        List<Double> macdSeries = new ArrayList<>();

        for (int i = 0; i < prices.size(); i++) {
            macdSeries.add(ema12.get(i) - ema26.get(i));
        }

        List<Double> signal = emaSeries(macdSeries, 9);
        int last = macdSeries.size() - 1;

        double macd = macdSeries.get(last);
        double signalValue = signal.get(last);

        return new double[]{macd, signalValue, macd - signalValue};
    }

    public double[] bollinger(List<CandleDto> candles, int period) {
        List<CandleDto> recent =
                candles.subList(candles.size() - period, candles.size());

        double middle = recent.stream()
                .mapToDouble(CandleDto::tradePrice)
                .average().orElse(0);

        double variance = recent.stream()
                .mapToDouble(c -> Math.pow(c.tradePrice() - middle, 2))
                .average().orElse(0);

        double std = Math.sqrt(variance);

        return new double[]{
                middle + std * 2,
                middle,
                middle - std * 2
        };
    }

    public double volumeMa(List<CandleDto> candles, int period) {
        return candles.subList(candles.size() - period, candles.size())
                .stream()
                .mapToDouble(CandleDto::volume)
                .average()
```

```
                .orElse(0);
    }
}
```

6. Kotlin - IndicatorService 핵심 함수

```kotlin
@Service
class IndicatorService {

    fun ma(candles: List<CandleDto>, period: Int): Double =
        candles.takeLast(period)
            .map { it.tradePrice }
            .average()

    fun ema(candles: List<CandleDto>, period: Int): Double {
        val k = 2.0 / (period + 1)
        var ema = candles.first().tradePrice

        candles.drop(1).forEach { candle ->
            ema = candle.tradePrice * k + ema * (1 - k)
        }
        return ema
    }

    fun wilderRsi(candles: List<CandleDto>, period: Int): Double {
        if (candles.size <= period) return 0.0

        var gain = 0.0
        var loss = 0.0

        for (i in 1..period) {
            val change = candles[i].tradePrice - candles[i - 1].tradePrice
            if (change > 0) gain += change
            else loss += kotlin.math.abs(change)
        }

        var avgGain = gain / period
        var avgLoss = loss / period

        for (i in period + 1 until candles.size) {
            val change = candles[i].tradePrice - candles[i - 1].tradePrice
            val currentGain = maxOf(change, 0.0)
            val currentLoss = maxOf(-change, 0.0)

            avgGain = (avgGain * (period - 1) + currentGain) / period
            avgLoss = (avgLoss * (period - 1) + currentLoss) / period
        }

        if (avgLoss == 0.0) return 100.0

        val rs = avgGain / avgLoss
        return 100 - (100 / (1 + rs))
    }

    fun emaSeries(values: List<Double>, period: Int): List<Double> {
        val result = mutableListOf<Double>()
        val k = 2.0 / (period + 1)
        var ema = values.first()
        result += ema

        values.drop(1).forEach { value ->
```

```
            ema = value * k + ema * (1 - k)
            result += ema
        }
        return result
    }

    fun macd(candles: List<CandleDto>): Triple<Double, Double, Double> {
        val prices = candles.map { it.tradePrice }
        val ema12 = emaSeries(prices, 12)
        val ema26 = emaSeries(prices, 26)

        val macdSeries = prices.indices.map { i ->
            ema12[i] - ema26[i]
        }

        val signal = emaSeries(macdSeries, 9)
        val macd = macdSeries.last()
        val signalValue = signal.last()

        return Triple(macd, signalValue, macd - signalValue)
    }

    fun bollinger(
        candles: List<CandleDto>,
        period: Int
    ): Triple<Double, Double, Double> {

        val recent = candles.takeLast(period)
        val middle = recent.map { it.tradePrice }.average()

        val variance = recent.map {
            val diff = it.tradePrice - middle
            diff * diff
        }.average()

        val std = kotlin.math.sqrt(variance)

        return Triple(
            middle + std * 2,
            middle,
            middle - std * 2
        )
    }

    fun volumeMa(candles: List<CandleDto>, period: Int): Double =
        candles.takeLast(period)
            .map { it.volume }
            .average()
}
```

7. 모든 지표를 한 번에 계산

Java - calculate()

```java
public IndicatorDto calculate(List<CandleDto> candles) {
    double price = candles.get(candles.size() - 1).tradePrice();

    double ma5 = ma(candles, 5);
    double ma20 = ma(candles, 20);
    double ema12 = ema(candles, 12);
    double ema26 = ema(candles, 26);
    double rsi = wilderRsi(candles, 14);

    double[] macd = macd(candles);
    double[] bb = bollinger(candles, 20);

    double volume = candles.get(candles.size() - 1).volume();
    double volumeMa20 = volumeMa(candles, 20);

    return new IndicatorDto(
        price, ma5, ma20, ema12, ema26, rsi,
        macd[0], macd[1], macd[2],
        bb[0], bb[1], bb[2],
        volume, volumeMa20
    );
}
```

Kotlin - calculate()

```kotlin
fun calculate(candles: List<CandleDto>): IndicatorDto {
    val price = candles.last().tradePrice
    val ma5 = ma(candles, 5)
    val ma20 = ma(candles, 20)
    val ema12 = ema(candles, 12)
    val ema26 = ema(candles, 26)
    val rsi = wilderRsi(candles, 14)

    val macd = macd(candles)
    val bb = bollinger(candles, 20)

    val volume = candles.last().volume
    val volumeMa20 = volumeMa(candles, 20)

    return IndicatorDto(
        price, ma5, ma20, ema12, ema26, rsi,
        macd.first, macd.second, macd.third,
        bb.first, bb.second, bb.third,
        volume, volumeMa20
    )
}
```

8. Gemma3에는 계산 결과만 전달

```text
예:
현재가격 4210
MA5 4198 / MA20 4130
EMA12 4185 / EMA26 4145
Wilder RSI14 63.7
MACD 40.2 / Signal 31.1 / Histogram 9.1
Bollinger Upper 4280 / Middle 4130 / Lower 3980
현재 거래량 3,400,000 / Volume MA20 2,100,000

→ 이 요약 숫자만 LangChain4j를 통해 Gemma3에 전달합니다.
```

9. Java - XrpAiService

```java
@Service
public class XrpAiService {

    private final ChatModel chatModel;

    public XrpAiService(ChatModel chatModel) {
        this.chatModel = chatModel;
    }

    public String analyze(IndicatorDto d) {

        double volumeRatio = d.volumeMa20() == 0
                ? 0
                : d.volume() / d.volumeMa20() * 100;

        String prompt = """
            당신은 XRP 기술적 분석 보조 AI입니다.
            아래 값은 Spring Boot가 계산한 결과입니다.

            현재가격: %.2f
            MA5: %.2f
            MA20: %.2f
            EMA12: %.2f
            EMA26: %.2f
            Wilder RSI14: %.2f
            MACD: %.4f
            Signal: %.4f
            Histogram: %.4f
            Bollinger Upper: %.2f
            Bollinger Middle: %.2f
            Bollinger Lower: %.2f
            현재 거래량: %.2f
            거래량 MA20: %.2f
            거래량 비율: %.1f%%

            추세:
            단기 방향: 상승 / 하락 / 횡보
            MA 분석:
            EMA 분석:
            RSI 분석:
            MACD 분석:
            볼린저밴드 분석:
            거래량 분석:
            종합 분석:
            위험요인:

            미래 가격을 확정적으로 예측하지 마세요.
            """.formatted(
                d.price(), d.ma5(), d.ma20(),
                d.ema12(), d.ema26(), d.rsi14(),
                d.macd(), d.macdSignal(), d.macdHistogram(),
                d.bollingerUpper(), d.bollingerMiddle(), d.bollingerLower(),
                d.volume(), d.volumeMa20(), volumeRatio
            );
```

```
        return chatModel.chat(prompt);
    }
}
```

10. Kotlin - XrpAiService

```kotlin
@Service
class XrpAiService(
    private val chatModel: ChatModel
) {
    fun analyze(d: IndicatorDto): String {

        val volumeRatio =
            if (d.volumeMa20 == 0.0) 0.0
            else d.volume / d.volumeMa20 * 100

        val prompt = """
            당신은 XRP 기술적 분석 보조 AI입니다.
            아래 값은 Spring Boot가 계산한 결과입니다.

            현재가격: ${d.price}
            MA5: ${d.ma5}
            MA20: ${d.ma20}
            EMA12: ${d.ema12}
            EMA26: ${d.ema26}
            Wilder RSI14: ${d.rsi14}
            MACD: ${d.macd}
            Signal: ${d.macdSignal}
            Histogram: ${d.macdHistogram}
            Bollinger Upper: ${d.bollingerUpper}
            Bollinger Middle: ${d.bollingerMiddle}
            Bollinger Lower: ${d.bollingerLower}
            현재 거래량: ${d.volume}
            거래량 MA20: ${d.volumeMa20}
            거래량 비율: $volumeRatio %

            추세:
            단기 방향: 상승 / 하락 / 횡보
            MA 분석:
            EMA 분석:
            RSI 분석:
            MACD 분석:
            볼린저밴드 분석:
            거래량 분석:
            종합 분석:
            위험요인:

            미래 가격을 확정적으로 예측하지 마세요.
        """.trimIndent()

        return chatModel.chat(prompt)
    }
}
```

11. Controller

Java

```java
@GetMapping("/predict")
public Map<String, Object> predict() {
    List<CandleDto> candles = upbitService.getXrpCandles();
    IndicatorDto indicators = indicatorService.calculate(candles);
    String analysis = aiService.analyze(indicators);

    return Map.of(
        "indicators", indicators,
        "analysis", analysis
    );
}
```

Kotlin

```kotlin
@GetMapping("/predict")
fun predict(): Map<String, Any> {
    val candles = upbitService.getXrpCandles()
    val indicators = indicatorService.calculate(candles)
    val analysis = aiService.analyze(indicators)

    return mapOf(
        "indicators" to indicators,
        "analysis" to analysis
    )
}
```

12. 실행 흐름과 핵심 정리

```text
GET http://localhost:8080/api/xrp/predict
        ↓
UpbitService → XRP 60분봉 200개
        ↓
IndicatorService → 모든 지표 계산
        ↓
IndicatorDto → 계산 결과 요약
        ↓
LangChain4j → Ollama → Gemma3
        ↓
지표별 해석 + 종합 분석
```

권장 설계: Java/Kotlin은 계산 엔진, Gemma3는 분석·설명 엔진으로 역할을 분리합니다. 이 방식은 캔들 원문 전체를 LLM에 보내는 것보다 입력량을 줄이고 계산을 코드로 통제할 수 있다는 장점이 있습니다.

다음 발전 단계는 SignalService를 추가하여 MA·EMA·RSI·MACD·볼린저밴드·거래량 조건별 점수를 계산한 뒤, 그 점수와 지표를 Gemma3가 설명하도록 만드는 것입니다.

주의: 기술지표와 LLM 분석은 미래 수익이나 가격을 보장하지 않으며 투자 판단의 보조 자료로만 사용해야 합니다.